multi-tenant-llm-router: a routing tier for multi-LoRA vLLM serving

Akshitha Reddy Lingampally

2026-06-06

Contents

1 Abstract

We present `multi-tenant-llm-router`, a FastAPI routing tier that sits in front of a vLLM `--enable-lora` server and adds the production concerns vLLM does not give you: per-tenant API key auth, per-tenant token-bucket rate limits, an LRU adapter cache, per-request structured JSONL logging, and per-tenant cost accounting. We report a load benchmark of 6 concurrent clients × 5 req/s × 15 seconds against the in-process mock backend: 429 requests total, 318 successful, 64.8% cache hit rate, p99 wall latency of 30 ms, 26% of requests rate-limited (by design, as the offered load of 30 req/s exceeds the configured 20 req/s tenant rate limit). All numbers are reproducible from a single `make bench` invocation; the mock backend lets the suite run without a GPU.

2 1. Background

vLLM (Kwon et al., 2023) is the production-default high-throughput LLM inference server, with first-class support for multi-LoRA serving via S-LoRA (Sheng et al., 2024). It does not, however, ship the routing-tier concerns that any multi-tenant production deployment needs:

- Authentication (which tenant is calling)
- Authorization (which adapters that tenant can use)
- Rate limits (per-tenant token-bucket)
- Cost accounting (per-tenant USD)
- Structured logs for billing + observability
- Adapter-cache bookkeeping (which adapters are currently warm)

This project is the routing layer that sits between clients and a real vLLM backend and adds those concerns. The actual GPU work stays in vLLM; this layer is what makes it safe to expose to many tenants.

3 2. Related Work

vLLM (Kwon et al., 2023): the inference server. PagedAttention is the technical contribution we build on top of.

S-LoRA (Sheng et al., 2024): multi-LoRA serving with thousands of concurrent adapters. The vLLM `--enable-lora` flag implements S-LoRA.

LoRA (Hu et al., 2022): the underlying parameter-efficient adaptation method.

4 3. Method

4.1 3.1 Pipeline

```
Client -> FastAPI router:
  1. Authenticate (Bearer key -> Tenant via constant-time hmac compare)
  2. Authorize  (req.adapter_id in tenant.allowed_adapters?)
  3. Rate limit (per-tenant token-bucket; refills at tenant.rate_limit_rps)
  4. Cache      (LRUAdapterCache.request -> was_cached, swap_ms)
  5. Backend    (vLLM completion call; mock for tests/CI)
  6. Log        (structured JSONL row to results/requests.jsonl)
  7. Return
```

4.2 3.2 Tenant config

YAML, one block per tenant; loaded at startup.

```yaml
t1:
  api_key: secret-t1
  allowed_adapters: [a0, a1, a2, ...]
  rate_limit_rps: 20.0
  cost_per_1k_tokens_usd: 0.002
```

API key check uses `hmac.compare_digest` (constant time, no prefix-leak side channel).

4.3 3.3 LRU adapter cache

The cache is bookkeeping-only: it tracks which adapter IDs would be warm if we were a real vLLM. Real GPU eviction is vLLM's job. On a hit we return the warm hit; on a miss we pay a configurable `mock_swap_ms` so the chart layer can show realistic swap-time distributions.

4.4 3.4 Token bucket

In-process, per-tenant, refills at `rate_limit_rps` tokens per second. Bucket capacity equals `rate_limit_rps` (so a burst of N requests at exactly N RPS sees them all admitted; the (N+1)th is rate-limited until next refill). Production should use a Redis-backed bucket for multi-replica deployment.

4.5 3.5 Mock backend

`MockBackend` simulates Poisson-distributed latency around a `base_latency_ms + per_token_ms * max_tokens` model. Used for CI and local development so the harness works without a GPU.

5 4. Data

Synthetic: load generator spawns N concurrent async clients (httpx), each emitting Poisson-distributed requests at `rps`. Each request picks an adapter at random from the tenant's allowed set, with 70% probability hitting the "hot" top-3 (Pareto-style distribution).

6 5. Evaluation Setup

Default load run: 6 concurrent clients × 5 req/s each × 15s duration, cache capacity = 4, 10 adapters with 70% traffic to top-3, mock backend. Hardware: Apple M-series CPU.

7 6. Results

metric	value
total requests	429
successful (200)	318
rate-limited (429)	111
cache hit rate	0.648
p50 wall latency	19.2ms
p95 wall latency	21.6ms
p99 wall latency	29.6ms
top-3 adapter share	75.5%
total cost (USD)	$0.0248

Two clean findings:

1. **111 of 429 requests (~26%) were rate-limited.** The fixture tenant `t1` has `rate_limit_rps = 20`, but 6 clients × 5 RPS = 30 RPS offered. The token-bucket correctly sheds the excess.
2. **Cache hit rate = 0.648 with capacity 4 and 10 adapters.** Because the top-3 get 70% of requests and the cache holds 4, those three plus one rotating slot stay warm. Bumping capacity to 6 would cover the top-6 and push hit rate over 0.85; that's the first lever to reach for.

8 7. Ablations

The cache-capacity sweep {2, 4, 6, 8, 10} confirms the expected monotonic hit rate increase. The 70-30 Pareto traffic pattern means the marginal return on capacity falls off after capacity = top-N hot adapters; capacity beyond the hot-adapter count buys negligible hit rate.

9 8. Discussion

The routing tier is what makes vLLM safe for multi-tenant production. The five concerns it adds (auth, authz, rate limit, cost, cache bookkeeping) are deliberately each small (~50-200 lines per concern); the value is in having all five together with consistent structured logging.

The mock-backend pattern is important: it lets the full request pipeline run in CI without a GPU, which means the test suite catches regressions in auth / rate limit / logging without spending GPU time.

10 9. Limitations

1. **In-process rate limit.** Per-process token bucket; multi-replica deployment needs Redis-backed (e.g. `slowapi` with Redis).
2. **No SSE pass-through.** vLLM supports streaming; the router currently buffers. Streaming pass-through is the obvious next add.
3. **Cache bookkeeping only.** The router doesn't actually call vLLM's `load_adapter` / `unload_adapter`; that integration is future work.
4. **Mock latency ≠ real vLLM.** The mock is fast (~20 ms) because it doesn't actually generate; real GPU vLLM is 50-200 ms.

11 10. Future Work

- ☐ Real vLLM backend client (POST /v1/load_adapter, /v1/completions).
- ☐ SSE pass-through for streaming completions.
- ☐ Prometheus `/metrics` endpoint.
- ☐ Redis-backed rate limiter for multi-replica.
- ☐ Per-adapter SLO config (route around hot adapters when p99 exceeds threshold).

12 11. References

- Hu, E. J., et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685.
- Kwon, W., et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* SOSP. arXiv:2309.06180.
- Sheng, Y., et al. (2024). *S-LoRA: Serving Thousands of Concurrent LoRA Adapters.* arXiv:2311.03285.

13 Appendix A. Reproducibility

- Repo: `Akshitha024/multi-tenant-llm-router`, MIT.

- Reproduce: `make serve` (in one terminal) + `make bench` + `make plots` (in another).
- Test artifacts in `docs/test_results/`.